Schéma de la base de données - pili-pos

Conformément au §II.C du Manuel de certification SCE 2.0, ce document liste **chaque table**, **chaque champ**, son **type**, sa **longueur** (le cas échéant) et sa **description**.

Moteur : SQLite 3.x bundled via `rusqlite` (Rust). **Encodage** : UTF-8. **Mode** : WAL (Write-Ahead Logging) - atomique crash-safe. **Fichier vivant** : `/var/lib/pili-pos/pos.db` en production. **Source de vérité du schéma** : `crates/pos-db/src/schema.rs`, `schema_hubrise.rs`, `schema_catalog.rs` et `schema_eft.rs` (lecture machine via `sqlite3 pos.db ".schema"`).

1. Tables de référence

1.1 `languages`

Langues prises en charge par les traductions multi-langues.

Champ	Type	Longueur	Description
`code`	TEXT	2	Clé primaire. Code ISO 639-1 : `fr`, `nl`, `de`, `en`, `zh`
`name`	TEXT	-	Nom de la langue dans cette langue (“Français”, “Deutsch”, “中文”…)
`active`	INTEGER	1 (booléen)	1 = langue activée pour ce déploiement, 0 = désactivée

Données initiales : `fr`, `nl`, `en`, `zh` insérées par la migration `001` ; `de` ajouté par la migration `007` (§6) pour qu’un catalogue en région de langue allemande puisse porter des désignations en allemand (la FK `translations.lang` → `languages.code` les refusait sinon) - voir `crates/pos-db/src/migrate.rs`.

1.2 `translations`

Table polymorphe : associe une chaîne traduite à n’importe quelle entité de la base (catégorie, item, modificateur…).

Champ	Type	Description
`id`	INTEGER	Clé primaire auto-incrémentée
`entity_type`	TEXT	Type d’entité : `category`, `menu_item`, `modifier_group`, `modifier`
`entity_id`	INTEGER	ID de l’entité concernée
`field`	TEXT	Nom du champ traduit : typiquement `name`, parfois `description`
`lang`	TEXT (FK → `languages.code`)	Code langue de la traduction
`value`	TEXT	Texte traduit

Contrainte d’unicité : `UNIQUE(entity_type, entity_id, field, lang)`. Index : `idx_translations_lookup(entity_t
entity_id, lang)`.

1.3 `categories`

Catégories de menu (ex. “Sushi”, “Boissons”).

Champ	Type	Description
`id`	INTEGER	Clé primaire
`external_id`	TEXT (unique)	Référence de la catégorie dans le catalogue externe (HubRise) ; NULL pour une catégorie créée à la caisse
`sort_order`	INTEGER	Ordre d’affichage
`color`	TEXT	Couleur hexadécimale optionnelle
`active`	INTEGER	Drapeau actif (0/1)
`hidden`	INTEGER (migration `021`)	1 = catégorie masquée de la grille de la caisse (ex. frais, articles réservés aux commandes en ligne), 0 par défaut

1.4 `menu_items`

Articles vendus. Chaque article appartient à exactement une catégorie et porte UN code TVA SCE 2.0 (A/B/C/D/X).

Champ	Type	Description
`id`	INTEGER	Clé primaire
`external_id`	TEXT (unique)	Référence de l’article dans le catalogue externe (HubRise) ; NULL pour un article créé à la caisse
`category_id`	INTEGER (FK → `categories.id`)	Catégorie parente
`price_cents`	INTEGER	Prix unitaire TVA incluse en centimes d’euro
`vat_code`	TEXT	Code TVA SCE en salle : `A`, `B`, `C`, `D`, `X`
`vat_category`	TEXT	Catégorie fiscale webshop : `food`, `beverage`, `zero_rated`, `out_of_scope`
`vat_code_takeaway`	TEXT (nullable, migration `002`)	Code TVA SCE à emporter, quand il diffère du code en salle (ex. `B`→`C`) ; NULL = le code en salle s’applique aussi à l’emporté
`open_price`	INTEGER (migration `002`)	1 = “prix libre” - l’opérateur saisit le prix unitaire à l’ajout (ex. produit 22010), 0 = prix fixe de `price_cents`
`kitchen_printer`	INTEGER	1 = à envoyer en cuisine, 0 = imprimante caisse seulement
`sort_order`	INTEGER	Ordre d’affichage dans la grille
`active`	INTEGER	1 = article disponible, 0 = retiré du menu
`kind`	TEXT (migration `021`)	`product` (défaut) ou `charge` (frais d’une commande en ligne : livraison, service) ; contraint

Champ	Type	Description
on_pos	INTEGER (migration 021)	1 (défaut) = affiché dans la grille de la caisse, 0 = réservé aux commandes en ligne

1.5 modifier_groups

Groupes de modificateurs liés à un article (ex. “Sauce” pour les sushi).

Champ	Type	Description
id	INTEGER	Clé primaire
external_id	TEXT (unique)	Référence dans le catalogue externe ; NULL pour un groupe créé à la caisse
min_select	INTEGER	Nombre minimum de modificateurs à sélectionner (défaut 0)
max_select	INTEGER	Nombre maximum (défaut 1)
menu_item_id	INTEGER (FK → menu_items.id)	Article parent
active	INTEGER (migration 021)	1 (défaut) = groupe en vente, 0 = retiré de la vente
sort_order	INTEGER (migration 021)	Ordre d’affichage

1.6 modifiers

Options individuelles d’un groupe (ex. “Sauce soja”).

Champ	Type	Description
id	INTEGER	Clé primaire
external_id	TEXT (unique)	Référence dans le catalogue externe ; NULL pour une option créée à la caisse
group_id	INTEGER (FK → modifier_groups.id)	Groupe parent
price_cents	INTEGER	Supplément de prix (peut être 0)
sort_order	INTEGER	Ordre dans le groupe
active	INTEGER (migration 021)	1 (défaut) = option en vente, 0 = retirée de la vente

Les noms des modificateurs sont dans translations (entity_type = modifier), pas dans cette table.

2. Tables transactionnelles

2.1 orders

Commandes (dine-in et takeaway). Une commande peut être ouverte (open), fermée (closed - encaissée) ou annulée (voided).

Champ	Type	Description
id	INTEGER	Clé primaire

Champ	Type	Description
table_number	INTEGER (nullable)	Numéro de table pour les dine-in
order_type	TEXT	dine_in ou takeaway
status	TEXT	open, closed ou voided
user_insz	TEXT	INSZ de l'opérateur qui a ouvert la commande
created_at	TEXT (ISO 8601)	Date / heure d'ouverture
closed_at	TEXT (nullable)	Date / heure d'encaissement
subtotal_cents	INTEGER	Sous-total TVA incluse
total_cents	INTEGER	Total payé en centimes
hubrise_order_id	TEXT (nullable)	Identifiant HubRise d'une commande en ligne ; NULL pour une commande de la caisse
is_fiscal	INTEGER	1 = commande soumise au FDM ; une commande en ligne passe à 1 dès sa signature (P ou N)
source	TEXT	pos_local (caisse), ou canal de la commande en ligne (par exemple deliveroo ; webshop sans canal)
hubrise_status	TEXT (nullable)	Statut HubRise de la commande en ligne
service_type	TEXT (nullable)	Livraison, enlèvement ou sur place (commande en ligne)
raw_hubrise_json	TEXT (nullable)	Commande en ligne telle que reçue de HubRise, **y compris les données du client** (nom, téléphone, adresse) ; NULL pour une commande de la caisse
pre_bill_at	TEXT (nullable, migration 009)	Horodatage de la dernière addition provisoire signée et imprimée (pre-bill), pour que l'écran des tables signale la table dont l'addition est posée ; remis à NULL dès qu'une ligne de la commande change

2.2 order_lines

Lignes d'une commande. Dénormalisées (le nom et le prix sont copiés depuis menu_items au moment de l'ajout, pour figer le tarif historique).

Champ	Type	Description
id	INTEGER	Clé primaire
order_id	INTEGER (FK → orders.id)	Commande parente
menu_item_id	INTEGER (FK → menu_items.id)	Article référencé
quantity	INTEGER	Quantité (peut être négative pour les remboursements)
unit_price_cents	INTEGER	Prix unitaire au moment de l'ajout (figé)
vat_code	TEXT	Code TVA appliqué (figé)
name	TEXT	Désignation imprimée (figée)

Champ	Type	Description
department_id	TEXT (nullable, migration 002)	Département SCE au moment de l'ajout, envoyé au FDM comme departmentId et totalisé dans les rapports X/Z
department_name	TEXT (nullable, migration 002)	Nom du département au moment de l'ajout, envoyé comme departmentName
modifiers_json	TEXT (nullable)	Modificateurs sélectionnés en JSON
created_at	TEXT	Date / heure d'ajout

order_lines reste la vue consolidée affichée/imprimée ; le journal chronologique des ajouts/corrections/remboursements est order_entries (§3.4).

2.3 payments

Paiements enregistrés pour une commande. Plusieurs paiements possibles par commande (espèces + carte sur un partage, titres-repas). Modes limités à cash, card, meal_voucher (migration 008) et other. Un mouvement de caisse (événement F) n'écrit aucune ligne payments.

Champ	Type	Description
id	INTEGER	Clé primaire
order_id	INTEGER (FK → orders.id)	Commande payée
method	TEXT	cash, card, meal_voucher ou other (contraint)
amount_cents	INTEGER	Montant en centimes (positif)
created_at	TEXT	Date / heure du paiement

2.4 table_plan, table_plan_log (migration 011)

Plan de salle : les numéros de table que l'établissement utilise, écrits en nombres et plages séparés par des virgules (1–12, 20–25), de 1 à 999, 200 tables au plus. Aucune donnée fiscale n'en dépend : le numéro de table figurait déjà sur la commande et ne change ni le ticket, ni la charge utile envoyée au FDM.

table_plan (une seule ligne, id = 1) :

Champ	Type	Description
id	INTEGER	Clé primaire, toujours 1 (CHECK(id = 1))
plan	TEXT	Plan en vigueur, sous sa forme canonique (1–12, 20–25)
config_plan	TEXT	Plan que pos.json (clé tables, ou 1 à table_count) donnait au dernier démarrage
changed_at	TEXT	Date du dernier changement
changed_by	TEXT (nullable)	INSZ du Gérant ; NULL pour un changement venu de pos.json (pas de clé étrangère)

table_plan_log (ajout seul) : une ligne par changement. Champs id, plan, source (config ou admin, contraint), changed_by (INSZ du Gérant, NULL pour config) et changed_at (défaut datetime('now')). Les déclencheurs table_plan_log_no_update et table_plan_log_no_delete refusent UPDATE et DELETE.

Même règle que l'interrupteur HubRise (§5.4) : au premier démarrage, ou quand `pos.json` donne un plan nouveau, ce plan s'applique et est journalisé (source `config`) ; sinon la dernière valeur reste, y compris celle qu'un Gérant a fixée dans Réglages (source `admin`). L'écran des tables affiche les tables du plan, plus toute table hors plan qui a encore une commande ouverte : aucune commande non réglée n'est donc masquée.

2.5 `eft_payments` Attention : APPEND-ONLY (migration `027`)

Paiements par carte demandés au terminal de paiement relié (réglage facultatif `payment_terminal` de `pos.json`). Une ligne par paiement demandé, écrite **avant** l'appel du terminal (statut `sent`) : un arrêt entre l'appel et la réponse laisse la trace qu'une carte a peut-être été débitée. La table n'est écrite que si l'installation a un terminal relié ; elle fait partie du binaire certifié de toute installation.

Champ	Type	Description
`id`	INTEGER	Clé primaire
`reference`	TEXT	Unique. UUID v4 envoyé au terminal (clé d'idempotence, clé de relecture du statut)
`order_id`	INTEGER (FK → `orders.id`)	Commande payée
`terminal_kind`	TEXT	`worldline_terminal_api`, `ccv_payment_api` ou `ccv_opi`
`amount_cents`	INTEGER	Montant demandé en centimes (strictement positif)
`status`	TEXT	`sent` (attend la réponse), `approved`, `declined`, `cancelled`, `unknown` (réponse perdue, à redemander) ou `not_sent` (rien n'a été débité) ; contraint
`card_type`	TEXT (nullable)	`debit`, `credit` ou `unknown` ; obligatoire pour un paiement `approved` (CHECK)
`brand`	TEXT (nullable)	Marque de la carte rendue par le terminal (par exemple Bancontact)
`terminal_reference`	TEXT (nullable)	Identifiant de transaction du terminal (devient la `reference` de la ligne de paiement signée)
`authorisation_code`	TEXT (nullable)	Code d'autorisation
`masked_pan`	TEXT (nullable)	Numéro de carte masqué par le terminal ; le numéro complet n'arrive jamais à la caisse
`customer_receipt`	TEXT (nullable)	Ticket client du terminal, conservé avec l'approbation
`detail`	TEXT (nullable)	Motif du refus, ce qui manque pour un paiement `unknown`, erreur d'un `not_sent`
`user_insz`	TEXT	INSZ de l'opérateur qui a demandé le paiement (pas de clé étrangère)
`created_at`	TEXT	Date / heure de la demande (défaut `datetime('now')`)
`resolved_at`	TEXT (nullable)	Date / heure de la réponse définitive
`used_event_id`	INTEGER (FK → `fiscal_events.id`, nullable)	La vente (événement N) qui a réglé ce paiement ; une seule fois

Index : idx_eft_payments_order (order_id) et idx_eft_payments_open (partiel, WHERE status IN ('sent','unknown'), les paiements qui attendent encore leur réponse).

Six déclencheurs (RAISE(ABORT, ...)), détaillés dans SECURITE_BD_FR.md : eft_payments_no_delete (jamais de DELETE), eft_payments_starts_sent (une ligne naît sent et non utilisée), eft_payments_request_fixed (ce qui a été demandé ne change jamais), eft_payments_status_moves (seuls sent et unknown évoluent, jamais vers sent), eft_payments_answer_final (une réponse approved, declined, cancelled ou not_sent ne change plus) et eft_payments_used_once (seul un paiement approved est utilisé, une fois).

3. Tables fiscales (append-only)

3.1 fiscal_events Attention : APPEND-ONLY

Le cœur du système fiscal. Une ligne par événement SCE 2.0 envoyé au FDM, jamais modifiée ni supprimée après insertion.

Champ	Type	Description
id	INTEGER	Clé primaire auto-incrémentée
event_type	TEXT	N, P, T, C, I, F, S, ou R (contraint par CHECK)
sequence_number	INTEGER	Compteur local POS (cf. STRATEGIE_COMPTEURS_FR.md)
order_id	INTEGER (FK → orders.id, nullable)	Commande associée pour N/P/I/C ; NULL pour F/S/R
pos_id	TEXT	Identifiant POS 14 caractères (cf. spec POS v2.2)
terminal_id	TEXT	ID du terminal physique
user_insz	TEXT	INSZ de l'opérateur
event_data_json	TEXT	Payload SCE complet sérialisé en JSON
fdm_response_json	TEXT (nullable)	Réponse brute du FDM (signature, counters, footer, QR)
fdm_id	TEXT (nullable)	ID du FDM extrait de la réponse
fdm_datetime	TEXT (nullable)	fdmDateTime extrait
event_label	TEXT (nullable)	Label retourné par le FDM (peut différer de event_type en formation)
short_signature	TEXT (nullable)	40 caractères hexadécimaux (SHA-1), événements N uniquement, imprimés sur le ticket
qr_data	TEXT (nullable)	URL 38 caractères pour le QR code
created_at	TEXT	Date / heure d'insertion (= clôture de la tx)
archived	INTEGER	Vestige de schéma - toujours 0, candidat à suppression

Contrainte d'unicité : UNIQUE(event_type, sequence_number) - garantit la **stricte séquentialité par type d'événement**.

3.2 fiscal_counters

Compteurs persistants par label d'événement.

Champ	Type	Description
event_label	TEXT	Clé primaire = label SCE (N, P, T, C, I, F, S, R)
current_value	INTEGER	Dernier compteur émis pour ce label

Modification exclusive via pos_fiscal::counter::FiscalCounter::next (atomique avec l'archive, cf. STRATEGIE_COMPTEURS_FR.md).

3.3 z_reports

Index des rapports Z générés. Référence le fiscal_events correspondant.

Champ	Type	Description
id	INTEGER	Clé primaire
report_date	TEXT	Date de la journée fiscale clôturée
sequence_number	INTEGER	Numéro de Z séquentiel
data_json	TEXT	Snapshot des agrégats au moment du Z
fiscal_event_id	INTEGER (FK → fiscal_events.id)	Lien vers l'événement R signé
created_at	TEXT	Date / heure de génération

3.4 order_entries (migration 002)

Journal append-only de chaque ajout, correction ou remboursement de ligne sur une commande ouverte (conforme POS v2.2 §2 "toutes les données d'entrée, par ordre chronologique, y compris les corrections"). order_lines reste la vue consolidée affichée/imprimée ; order_entries porte la trace chronologique transmise au FDM.

Champ	Type	Description
id	INTEGER	Clé primaire, ordre chronologique
order_id	INTEGER (FK → orders.id)	Commande parente
line_id	INTEGER	Ligne d'origine référencée
menu_item_id	INTEGER	Article
quantity	INTEGER (≠ 0)	Quantité de l'entrée (négative pour correction/remboursement)
unit_price_cents	INTEGER	Prix unitaire au moment de l'entrée
vat_code	TEXT	Code TVA (contraint A/B/C/D/X)
name, department_id, department_name	TEXT	Désignation et département au moment de l'entrée
kind	TEXT	entry, correction ou refund (contraint)
sent_event_id	INTEGER (FK → fiscal_events.id, nullable)	Rempli quand l'entrée a été transmise dans un événement P ou dans le N final
created_at	TEXT	Date / heure d'ajout

Deux triggers SQL font respecter l'append-only :

- order_entries_no_delete - BEFORE DELETE, avorte systématiquement (RAISE(ABORT, ...)).

- `order_entries_no_rewrite` - `BEFORE UPDATE` sur toutes les colonnes journalisées (`order_id`, `line_id`, `menu_item_id`, `quantity`, `unit_price_cents`, `vat_code`, `name`, `department_id`, `department_name`, `kind`, `created_at`), avorte systématiquement. Seule `sent_event_id` reste modifiable - elle passe de `NULL` à l'`id` de l'événement fiscal une fois l'entrée effectivement transmise au FDM, sans quoi elle ne serait jamais renseignable.

Index : `idx_order_entries_order(order_id, id)`.

3.5 `booking_periods` (migration `003`)

Persistance de la période d'ouverture (`bookingPeriodId`, POS v2.2 §2.2.2). Une période court du premier événement suivant un Z jusqu'au Z suivant, et porte la date d'ouverture (fuseau Europe/Brussels). Voir §III.J et §III.L de `REPONSES_MANUEL_CERT_FR.md`.

Champ	Type	Description
`id`	TEXT	Clé primaire (UUID v4, générée par `commands::booking_period`)
`booking_date`	TEXT	Date d'ouverture, fuseau Europe/Brussels
`opened_at`	TEXT	Horodatage d'ouverture
`closed_at`	TEXT (nullable)	Horodatage de clôture (NULL = période ouverte)
`closing_event_id`	INTEGER (FK → `fiscal_events.id`, nullable)	Événement R qui a clôturé la période (NULL tant qu'ouverte)

Contrainte `CHECK ((closed_at IS NULL) = (closing_event_id IS NULL))` - les deux colonnes sont renseignées ou nulles ensemble. Au plus une période est ouverte à la fois, via l'index unique partiel `idx_booking_periods_one_open` sur l'expression `(closed_at IS NULL)` filtrée `WHERE closed_at IS NULL` (deux lignes ouvertes collisionneraient sur la même valeur d'expression). Deux triggers :

- `booking_periods_no_reopen` - `BEFORE UPDATE ... WHEN OLD.closed_at IS NOT NULL`, avorte toute modification d'une période déjà close (pas seulement sa réouverture).
- `booking_periods_no_delete` - `BEFORE DELETE`, avorte systématiquement.

3.6 `ticket_deliveries` Attention : APPEND-ONLY (partiel)

File d'envoi et journal d'audit du ticket numérique par e-mail (migration `005`, cf. §III.H de `REPONSES_MANUEL_CERT_FR.md` et le détail complet dans `TICKET_NUMERIQUE_FR.md`). Une ligne par ticket dont le support choisi inclut l'e-mail (`DIGITAL` ou `PAPER_DIGITAL`), ajoutée dans la **même transaction** que l'événement fiscal qu'elle accompagne.

Champ	Type	Description
`id`	INTEGER	Clé primaire
`fiscal_event_id`	INTEGER (FK → `fiscal_events.id`)	Événement dont ce ticket est la copie numérique. **Immuable** (trigger).
`medium`	TEXT	`DIGITAL` (e-mail seul, papier en secours) ou `PAPER_DIGITAL` (papier toujours + e-mail en bonus). Contraint. **Immuable**.
`recipient`	TEXT (nullable)	Adresse e-mail en clair. `NULL` dès l'envoi réussi ou le repli papier, et au plus tard 72 h après création (purge RGPD).

Champ	Type	Description
recipient_hint	TEXT	Adresse masquée (ex. j***@e***.be), conservée indéfiniment pour la lisibilité du support/dossier.
ticket_text	TEXT	Texte du ticket, identique à ce qui s'imprime. **Immuable**.
verification_url	TEXT (nullable)	URL de vérification FDM ; NULL pour les copies (C) et tickets de formation (T), qui n'en reçoivent pas.
status	TEXT	pending, sending, sent, failed, fallback_printed, ou purged. Contraint.
attempts	INTEGER	Nombre de tentatives d'envoi effectuées
next_attempt_at	TEXT (nullable)	Prochaine échéance de réessai (recul exponentiel)
provider_message_id	TEXT (nullable)	Identifiant renvoyé par Scaleway TEM sur succès
last_error	TEXT (nullable)	Dernière erreur d'envoi
fallback_printed	INTEGER	1 = le papier de secours a été imprimé pour ce ticket
created_at / updated_at	TEXT	Horodatages

Triggers : suppression interdite, et modification interdite de fiscal_event_id / ticket_text / medium (les autres colonnes, notamment status, restent modifiables - c'est le cycle de vie de la file). Index : idx_ticket_deliveries_due(status, next_attempt_at) pour le worker de réessai.

3.7 original_prints Attention : APPEND-ONLY

Une ligne par événement fiscal dont l'**original** a été imprimé (migration 006, POS spec v2.2 §8.1 : un original n'est imprimé qu'une seule fois). Sert de garde avant toute impression papier d'origine (checkout, remboursement, ou rattrapage d'un envoi numérique manqué) ; toute reproduction ultérieure passe par l'événement C (copie).

Champ	Type	Description
fiscal_event_id	INTEGER (FK → fiscal_events.id)	Clé primaire - au plus une ligne par événement
printed_at	TEXT	Horodatage de l'impression

Triggers : suppression et modification interdites (table strictement append-only, sans exception).

3.8 users

Utilisateurs du POS. INSZ comme clé primaire. Voir PROFILS_UTILISATEURS_FR.md.

Champ	Type	Description
insz	TEXT (11 chiffres)	Clé primaire. Numéro National Belge
name	TEXT	Nom affichable

Champ	Type	Description
role	TEXT	admin, cashier, ou waiter (contraint)
pin_hash	TEXT (nullable)	Hash bcrypt du PIN optionnel
active	INTEGER	1 = utilisateur actif, 0 = désactivé
presence_only	INTEGER	0 par défaut, 1 = personnel sans accès à la caisse (profil « Présence seule », PROFILS_UTILISATEURS_FR.md §2.4) : pas de PIN, pas de connexion, pointé par un Gérant. CHECK 0 ou 1 (migration 028)

3.8 bis staff_shifts (migration 009, colonnes d'audit ajoutées par 028)

Index local des postes de travail : une ligne par prestation, ouverte par un événement S WorkIn signé et close par un S WorkOut signé, dans la même transaction que l'événement. Verrouiller l'écran laisse le poste ouvert, ce qui permet à l'écran de connexion d'afficher qui est en service et depuis quand. Le registre fiscal reste l'événement S ; cette table n'en est qu'un index. Elle n'est pas append-only (ended_at est mis à jour).

Champ	Type	Description
id	INTEGER	Clé primaire
user_insz	TEXT	INSZ de l'employé (pas de clé étrangère)
started_at	TEXT (ISO 8601, UTC)	Début du poste
ended_at	TEXT (nullable)	Fin du poste ; NULL = poste ouvert
started_by	TEXT (nullable)	INSZ de la personne qui a enregistré l'arrivée : l'employé lui-même (connexion) ou le Gérant qui l'a pointé ; NULL pour les lignes d'avant la migration 028 (migration 028)
ended_by	TEXT (nullable)	Idem pour le départ (migration 028)

Index : idx_staff_shifts_open(user_insz, ended_at). Au plus un poste ouvert par employé (la requête d'ouverture ne crée rien s'il en existe un). Un événement S d'entraînement (étiquette T) n'ouvre ni ne ferme jamais de poste : l'index ne reflète que la présence réelle.

3.9 pending_signs (migration 010)

Requête de signature stockée **avant** son envoi au FDM, pour pouvoir la renvoyer à l'identique si la réponse est perdue (voir *STRATEGIE_COMPTEURS_FR.md* §5). La base est en synchronous=FULL : la ligne est durable avant l'envoi.

Champ	Type	Description
id	INTEGER PK	Identifiant
event_label	TEXT	Label de l'événement (N, R, P, …)
first_ticket_no	INTEGER	Premier numéro de ticket de la requête

Champ	Type	Description
ticket_count	INTEGER	1, ou 2 pour la paire Z chiffre d'affaires + Z utilisateur (M184)
mutation	TEXT	Mutation FDM envoyée
is_training	INTEGER	0 ou 1
request_body	TEXT	Octets exacts de la requête, renvoyés tels quels
archive_json	TEXT	Événements à archiver à la réponse
kind	TEXT	Opération (sale, refund, report_z, ...)
subject	TEXT	Objet de l'opération (commande, vente remboursée, ...)
order_id	INTEGER (nullable)	Commande concernée
context_json	TEXT	Effets métier à appliquer (commande clôturée, paiements, livraison, rapport)
user_insz	TEXT	Opérateur
status	TEXT	open (défaut), signed, refused, not_sent
attempts	INTEGER	Nombre d'envois
last_error	TEXT (nullable)	Dernière erreur
fiscal_event_id	INTEGER (FK → fiscal_events.id, nullable)	Événement archivé (premier événement si ticket_count = 2)
created_at, resolved_at	TEXT	Horodatages

Index : - idx_pending_signs_one_open (unique partiel, WHERE status = 'open') : au plus une ligne ouverte à la fois. - idx_pending_signs_order sur (order_id, status) : retrouver une commande dont la vente attend la réponse du FDM.

Triggers : - pending_signs_request_immutable : ce qui a été envoyé (label, numéros, corps, événements, opération, opérateur, created_at) ne change jamais. - pending_signs_context_on_resolution : context_json ne change qu'à la sortie de l'état open (l'adresse e-mail d'un ticket numérique y est alors effacée). - pending_signs_no_reopen : une ligne résolue ne change plus. - pending_signs_no_delete : suppression interdite. Une ligne ouverte ne se supprime jamais à la main : voir la procédure de dépannage du *MANUEL_INSTALLATION_FR.md*.

3.10 Garde de période : fiscal_events_period_open (migration 010)

Déclencheur BEFORE INSERT sur fiscal_events : refuse (booking period is closed, code applicatif period_closed) l'archivage d'un événement dont le bookingPeriodId désigne une période d'ouverture déjà fermée (booking_periods.closed_at non nul). Un événement signé ne peut donc pas être rattaché à un jour déjà clôturé.

4. Tables Peppol (hors binaire certifié v1.0.0)

4.1 peppol_deliveries

Audit trail des factures B2B livrées via Access Point Peppol. La table est créée par la migration 1 dans toute build, mais le code Peppol (#[cfg(feature = "peppol")]) n'est pas dans le binaire certifié de la v1.0.0 : aucune ligne n'y est jamais écrite.

Champ	Type	Description
id	INTEGER	Clé primaire
fiscal_event_id	INTEGER (FK)	Lien vers l'événement I correspondant
invoice_no	TEXT	Numéro de facture émetteur
customer_endpoint_id	TEXT	Identifiant Peppol du destinataire (0208:BE...)
ap_vendor	TEXT	Nom du fournisseur AP ("storecove", "unifiedpost", "logging-only")
ap_message_id	TEXT (nullable)	ID retourné par l'AP
ubl_xml	TEXT	Document UBL 2.1 complet pour audit
ap_response_json	TEXT (nullable)	Réponse de l'AP
status	TEXT	queued, accepted, ou failed (contraint)
error_msg	TEXT (nullable)	Message d'erreur si failed
created_at	TEXT	Date / heure de l'envoi
accepted_at	TEXT (nullable)	Date / heure d'acceptation par l'AP

Index : idx_peppol_deliveries_invoice(invoice_no).

5. Tables des commandes en ligne (migrations 20 à 25)

Présentes sur toute installation ; écrites seulement quand les commandes en ligne sont activées pour l'installation (COMMANDES_EN_LIGNE_FR.md §2), sauf le journal de l'interrupteur.

5.1 webshop_orders

Une ligne par commande en ligne, à côté de sa ligne orders.

Colonne	Type	Description
order_id	INTEGER PK	Référence orders(id)
hubrise_order_id	TEXT UNIQUE	Identifiant HubRise ; reference du costCenter
channel / channel_kind	TEXT	Canal (par exemple deliveroo) ; platform ou webshop
service_type	TEXT	delivery, collection ou eat_in
payment_mode / payment_type	TEXT	prepaid ou unpaid ; APP ou ONLINE pour une commande payée
fiscal_state	TEXT	pending, signing, signed_p, signed_n, voided, rejected, refund_pending, cancel_pending, refunded, closed
resolution	TEXT	ok ou needs_review (« à vérifier »)
attention_json / warnings_json	TEXT (JSON)	Raisons du contrôle, avertissements

Colonne	Type	Description
lines_json	TEXT (JSON)	Lignes à signer, rapprochées du menu
p_event_id / n_event_id / refund_event_id	INTEGER	Référencent fiscal_events(id)
accepted_by / accepted_at	TEXT	INSZ de l'employé qui a accepté, date
resolved_at	TEXT (défaut datetime('now'))	Date du dernier rapprochement de la commande avec le menu de la caisse ; réécrit à chaque nouveau rapprochement tant que fiscal_state = pending

Index : idx_webshop_orders_state(fiscal_state, resolution).

5.2 hubrise_channels, hubrise_cash_payment_refs

hubrise_channels associe un canal à son type (platform / webshop), au type de paiement FDM d'une commande payée (APP / ONLINE) et au traitement de ses frais (charge_policy : include ou exclude, exclude par défaut pour les plateformes depuis la migration 24). hubrise_cash_payment_refs liste les références de paiement HubRise qui désignent des espèces.

5.3 order_price_changes (ajout seul)

Remises d'une commande en ligne, envoyées au FDM comme priceChanges (spécification POS v2.2). Une ligne par remise et par ligne du journal (entry_id référence order_entries(id)) : change_id, name, scope (LINE / EVENT), change_type (PUBLIC / INTERNAL), amount_cents. line_id est l'identifiant de la ligne d'order_lines concernée (sans clé étrangère) ; grouping_id (> 0) regroupe une remise appliquée à plusieurs lignes à la fois, les remises successives l'incrémentant (FAQ producteurs Q304). Index idx_order_price_changes_order(order_id, entry_id). Déclencheurs refusant UPDATE et DELETE.

5.4 integration_switches, integration_switch_log (ajout seul)

integration_switches : état de l'interrupteur (name = hubrise, enabled, config_value = valeur de pos.json au dernier démarrage, changed_at, changed_by). integration_switch_log : chaque changement (enabled, source = config ou admin, changed_by = INSZ du Gérant, changed_at). Déclencheurs refusant UPDATE et DELETE.

5.5 Liaison HubRise

hubrise_connection (une seule ligne, id = 1) : connexion de l'installation à son établissement HubRise.

Colonne	Type	Description
access_token	TEXT	Jeton d'accès OAuth HubRise, **en clair** (voir SECURITE_BD_FR.md §5)
account_id / account_name	TEXT	Compte HubRise
location_id / location_name	TEXT	Établissement HubRise
catalog_id / catalog_name	TEXT	Catalogue lié
customer_list_id / customer_list_name	TEXT (nullable)	Liste de clients liée
scope	TEXT	Portée OAuth accordée
base_url	TEXT	URL de l'API HubRise

Colonne	Type	Description
`connected_at`	TEXT	Date de connexion
`last_poll_at`	TEXT (nullable)	Dernière lecture des événements
`last_event_cursor`	TEXT (nullable)	Curseur des événements
`last_catalog_sync_at`	TEXT (nullable)	Dernière synchronisation du catalogue
`auth_error_at`	TEXT (nullable, migration `020`)	Date du dernier refus d'autorisation
`last_error`	TEXT (nullable, migration `020`)	Dernière erreur de transport
`callback_registered_at`	TEXT (nullable, migration `020`)	Date d'enregistrement du rappel

`hubrise_events` : événements reçus (idempotence par l'identifiant HubRise).

Colonne	Type	Description
`id`	TEXT PK	Identifiant de l'événement HubRise
`resource_type` / `event_type`	TEXT	Ressource et type d'événement
`payload_json`	TEXT	Contenu reçu
`received_at` / `processed_at`	TEXT	Réception ; traitement (NULL = à traiter)
`error_msg`	TEXT (nullable)	Dernière erreur de traitement
`attempts`	INTEGER (migration `020`)	Nombre de tentatives de traitement
`parked_at`	TEXT (nullable, migration `020`)	Mis de côté après échecs répétés

Index : `idx_hubrise_events_pending(received_at) WHERE processed_at IS NULL`.

`hubrise_outbox` : messages à envoyer à HubRise (création d'une vente de la caisse, changement de statut), dans l'ordre par commande.

Colonne	Type	Description
`id`	INTEGER PK	Identifiant
`order_id`	INTEGER	Commande de la caisse
`body_json`	TEXT	Corps du message
`created_at`	TEXT	Mise en file
`attempts` / `last_attempt_at`	INTEGER / TEXT	Tentatives d'envoi
`sent_at`	TEXT (nullable)	Envoi réussi (NULL = en file)
`error_msg`	TEXT (nullable)	Dernière erreur
`kind`	TEXT (migration `020`)	`create_order` (défaut) ou `patch_order` ; contraint
`hubrise_order_id`	TEXT (nullable, migration `020`)	Commande HubRise visée par un `patch_order`
`private_ref`	TEXT (nullable, migration `020`)	Référence privée dédoublonnant les créations
`next_attempt_at` / `parked_at`	TEXT (nullable, migration `020`)	Prochaine tentative ; mis de côté

Index : `idx_hubrise_outbox_pending(created_at) WHERE sent_at IS NULL` ; `idx_hubrise_outbox_private_ref(p` (unique, partiel) ; `idx_hubrise_outbox_order(hubrise_order_id, id) WHERE sent_at IS NULL`.

Les ventes de la caisse ne sont mises en file (`create_order`) que si l'intégration est activée pour l'installation.

5.6 Menu reflet : `catalog_sync_state` (migration `021`)

Une ligne par source de catalogue.

Colonne	Type	Description
`source`	TEXT PK	Source du catalogue (ex. `hubrise`)
`manages_menu`	INTEGER	1 = cette source gère le menu, l'édition locale est verrouillée
`hash`	TEXT (nullable)	Empreinte du dernier catalogue
`applied_at` / `rejected_at`	TEXT (nullable)	Dernier catalogue appliqué ; dernier refusé
`issues_json`	TEXT (nullable)	Problèmes relevés au dernier refus
`catalog_json`	TEXT (nullable)	Dernier catalogue brut reçu

La migration 21 ajoute aussi `categories.hidden`, `menu_items.kind`, `menu_items.on_pos`, `modifier_groups.active` / `sort_order` et `modifiers.active` (§1.3 à 1.6). La synchronisation met à jour ces lignes en place (jamais de suppression) ; l'historique fiscal est figé dans `order_lines` et `order_entries`.

6. Migrations

Système de migrations versionné dans `crates/pos-db/src/migrate.rs`. À la version actuelle, **vingt migrations** existent (1 à 11, puis 20 à 28 ; 12 à 19 sont réservées aux correctifs du binaire certifié), appliquées dans l'ordre et consignées dans la table `_migrations_applied` (`version`, `name`, `applied_at`) :

Version	Nom	Contenu
1	`initial-schema`	Crée l'ensemble des tables listées au §§1-5 ci-dessus.
2	`catalogue-fiscal-fields`	Ajoute `vat_code_takeaway` et `open_price` sur `menu_items`, `department_id`/`department_name` sur `order_lines`, et la table `order_entries` avec ses triggers (§3.4).
3	`booking-periods`	Ajoute la table `booking_periods` avec ses triggers et son index (§3.5).
4	`fiscal-archive-append-only`	Ajoute les triggers interdisant `DELETE`/`UPDATE` sur `fiscal_events` et `z_reports` (§3.1, §3.3).
5	`ticket-deliveries`	Ajoute la table `ticket_deliveries` (§3.6, ticket numérique par e-mail).
6	`original-prints`	Ajoute la table `original_prints` (§3.7, impression unique de l'original).
7	`german-language`	Insère `de` (Deutsch) dans `languages`, pour qu'un catalogue en région de langue allemande puisse porter des désignations en allemand (§1.1).

Version	Nom	Contenu
8	`meal-voucher-payments`	Autorise le moyen de paiement `meal_voucher` dans `payments` (table reconstruite avec ses lignes, SQLite ne modifiant pas un CHECK).
9	`shifts-and-pre-bill`	Ajoute `staff_shifts` (index local des prestations ouvertes par un S WorkIn) et `orders.pre_bill_at`.
10	`pending-signs`	Ajoute `pending_signs` (requêtes de signature stockées avant envoi, §3.9) et le déclencheur `fiscal_events_period_open` (§3.10).
11	`table-plan`	Ajoute `table_plan`, `table_plan_log` et ses déclencheurs (§2.4, plan de salle).
20	`hubrise-transport`	Colonnes de nouvelle tentative et d'erreur sur `hubrise_connection`, `hubrise_events`, `hubrise_outbox` (§5.5).
21	`catalog-sync`	Menu reflet d'un catalogue externe, `catalog_sync_state` (§5.5).
22	`webshop-orders`	`webshop_orders`, `hubrise_channels`, `hubrise_cash_payment_refs` (§5.1, §5.2).
23	`price-changes`	`order_price_changes` et ses déclencheurs (§5.3).
24	`platform-charges`	Frais des plateformes exclus par défaut (§5.2).
25	`integration-switch`	`integration_switches`, `integration_switch_log` et ses déclencheurs (§5.4).
26	`product-codes`	Colonne `code` de `menu_items` (code de menu affiché, sans effet fiscal).
27	`eft-payments`	`eft_payments`, ses index et ses six déclencheurs (§2.5, terminal de paiement relié).
28	`presence-registration`	Colonne `users.presence_only` (profil « Présence seule », §3.8) et colonnes d'audit `staff_shifts.started_by` / `ended_by` (§3.8 bis) : le Gérant qui pointe un autre employé. Non conditionnée par une fonctionnalité de compilation.

Chaque migration s'exécute dans sa propre transaction `BEGIN IMMEDIATE` ; un échec entraîne un `ROLLBACK` complet de cette migration sans toucher les précédentes déjà consignées.

Les bases “legacy” créées avant la versification sont automatiquement annotées comme `v1 (backfilled)`.

7. Méthode d’export pour audit

Voir FEUILLE_ACQUISITION_DONNEES_FR.md §2.